

08 — Full Retest + Manual QA + Release Tagging

Revision: 1 **Parent:** 00_MASTER_INDEX.md **Prerequisites:** All prior stages (B through G) must be complete before H-09 (§11.4.185 Manual QA).

Overview

This is the FINAL GATE. Nothing is “production” until H-09 passes. The full test suite must be GREEN across all components. Every feature must have video recording evidence. An independent code review must reach zero-finding GO. And a MANUAL QA team confirmation (§11.4.185) must occur on real hardware as the absolute final step.

H-01 [AGENT] — Full Server Test Suite Re-Run

Effort: M (~1h automated, ~1h review)

What to run:

```
cd server

# Build + vet
go build ./...
go vet ./...
gofmt -l . | grep -v vendor # should be empty

# Unit tests (all packages)
go test -race -count=1 ./... 2>&1 | tee /tmp/server-unit.log

# Integration tests (requires PostgreSQL)
```

```

POSTGRES_TEST=1 go test -tags integration -race -count=1 ./
    internal/store/... ./internal/rollout/... 2>&1 | tee /tmp/
    server-integration.log

# Stress tests
go test -tags stress -count=1 ./internal/api/... 2>&1 | tee /tmp/
    server-stress.log

# Chaos tests
go test -tags chaos -count=1 ./internal/api/... ./internal/
    store/... 2>&1 | tee /tmp/server-chaos.log

# Fuzz tests (short run – 30s each)
for fuzz in $(grep -rl "func Fuzz" internal/); do
    go test -fuzz=. -fuzztime=30s $(dirname $fuzz)
done

# Security tests
go test -tags security -count=1 ./internal/api/... 2>&1 | tee /
    tmp/server-security.log

# Race detector (re-run with higher count)
go test -race -count=5 ./internal/api/... ./internal/store/...
    2>&1 | tee /tmp/server-race.log

# Memory tests
go test -tags memory -count=1 ./internal/api/... 2>&1 | tee /tmp/
    server-memory.log

# Benchmark baseline
go test -bench=. -benchmem ./internal/api/... ./internal/
    store/... ./internal/rollout/... 2>&1 | tee /tmp/server-
    bench.log

```

Acceptance criteria:

- go build exit 0
- go vet exit 0
- gofmt -l empty
- ALL test targets GREEN (0 failures)
- -race reports 0 data races
- Memory test confirms no leaks
- Benchmarks within 2x of baseline

Evidence:

- All logs committed to docs/qa/<date>-full-retest-server/

- Summary report: docs/qa/<date>-full-retest-server/REPORT.md
-

H-02 [AGENT] — Full OTA Brick Test Suite Re-Run

Effort: M (~30 min automated, ~30 min review)

What to run:

```
# ota-protocol
cd submodules/ota-protocol
go test -race -count=1 ./...
go test -fuzz=. -fuzztime=30s ./...

# ota-artifact-validator
cd ../ota-artifact-validator
go test -race -count=1 ./...
go test -fuzz=. -fuzztime=30s ./...

# ota-rollout-engine
cd ../ota-rollout-engine
go test -race -count=1 ./...
go test -tags stress -count=1 ./...
go test -tags chaos -count=1 ./...

# ota-telemetry-schema
cd ../ota-telemetry-schema
go test -race -count=1 ./...
go test -fuzz=. -fuzztime=30s ./...

# http3
cd ../http3
go test -race -count=1 ./...
```

Acceptance criteria:

- ALL test targets GREEN (0 failures in all 5 bricks)
 - -race clean across all bricks
 - Fuzz: 0 crashes in 30s × each fuzz target
-

H-03 [AGENT] — End-to-End Signed Artifact Pipeline Against Production

Effort: M (~1h)

What to run:

```
# Full E2E test against the production server
```

```
bash tests/e2e/pipeline_signed.sh https://hxota.dev
```

```
# Staged-rollout HALT safety test
```

```
bash tests/e2e/rollout_halt_safety.sh https://hxota.dev
```

```
# Recall lifecycle test
```

```
bash tests/e2e/recall_lifecycle.sh https://hxota.dev
```

```
# Telemetry schema live validation
```

```
bash tests/e2e/telemetry_schema_live.sh https://hxota.dev
```

```
# Security probes
```

```
bash tests/security/security_probes.sh https://hxota.dev
```

```
bash tests/security/security_probes_extended.sh https://hxota.dev
```

```
bash tests/security/trust_boundary_live.sh https://hxota.dev
```

```
bash tests/security/saturation_live.sh https://hxota.dev
```

Acceptance criteria:

- All E2E scripts PASS against the LIVE production server
 - Signed artifacts are accepted, unsigned are rejected
 - Rollout HALT triggers correctly on breach
 - Recall correctly terminates active deployment
 - Telemetry schema matches server expectations
-

H-04 [AGENT/HARDWARE] — Android Agent Real-Device Tests

Effort: M (~1h)

What to run:

```
cd submodules/ota-android-agent

# JVM unit tests
./gradlew :core:test
./gradlew :android:test

# Instrumentation tests (requires device)
./gradlew :android:connectedAndroidTest

# Manual stress: force rapid poll cycles
# Observe: no crashes, no ANRs, no memory growth
```

Acceptance criteria:

- All JVM tests GREEN
 - connectedAndroidTest GREEN on real RK3588 device
 - Stress manual: 100+ poll cycles, no regression
-

H-05 [AGENT] — Distributed DDoS Resilience Test

Effort: M (~1h) **Source:** PRODUCTION_READINESS_PLAN.md §3 (testing gap)

What to run:

```
# Multi-source load test against the production stack
# Use the loadtest tool from server/tools/loadtest
cd server
go run tools/loadtest/main.go \
  --target https://hxota.dev \
  --concurrency 50 \
  --duration 60s \
  --endpoints /healthz,/api/v1/devices,/api/v1/client/update \
  --ramp-up 10s

# Slowloris-style test
# Verify the rate limiter drops connections above threshold
```

Acceptance criteria:

- Server stays responsive during load (p95 latency < 500ms at 50 concurrent)
 - Rate limiter activates when threshold exceeded (429 responses)
 - No crashes, OOM, or degraded /healthz responses
 - Recovery after load ends (latency returns to baseline within 10s)
-

H-06 [AGENT] — Web-Surface Stress/Chaos Tests

Effort: M (~1h) **Source:** PRODUCTION_READINESS_PLAN.md §3

What to run:

```
# Dashboard Playwright e2e + host-render
cd dashboard
pnpm test:run
pnpm exec playwright test

# ota-manager vitest + Playwright
cd clients/ota-manager
pnpm test:run
pnpm exec playwright test

# §11.4.170 host-render verification
# All screens × {light,dark} × {loaded,empty,error}
bash scripts/collect_feature_evidence.sh
```

Acceptance criteria:

- Dashboard: vitest 100% PASS, Playwright 100% PASS, host-render all golden-match
 - ota-manager: vitest 100% PASS, Playwright 100% PASS, host-render all golden-match
 - a11y: axe-core 0 critical/serious violations on both apps
-

H-07 [AGENT] — Complete §11.4.153 Per-Feature Video Recording Evidence

Effort: L (~3h) **Source:** Constitution §11.4.153, PRODUCTION_READINESS_PLAN.md OTA-061

What to record:

Every feature row in docs/features/Status.md must have a durable, committed MP4 recording that is: - Window-scoped (captures only the app window, not whole desktop) — §11.4.154 - Project-prefixed filename (helix_ota---*.mp4) — §11.4.155 - Vision-verified (OCR/content read-back confirms expected results) — §11.4.158/§11.4.159 - Durable (committed to the repo, not ephemeral) — §11.4.153

Features needing (re-)recording:

1. Server: API endpoints (CRUD for all resources), auth flow, health probes
2. Dashboard: Login, devices, artifacts, releases, deployments, rollouts, groups, telemetry, audit
3. ota-manager: Login, dashboard, devices, artifacts, etc.
4. Emulator: Tier-1 container OTA cycle
5. QEMU: A/B slot switch, rollback
6. Production: Full OTA cycle on real hardware (G-08 recording)

Process per feature:

1. **SPECIFY:** What should the recording show (expected content specification BEFORE recording)
 2. **RECORD:** Capture window-scoped MP4
 3. **VALIDATE:** Run through vision/OCR bridge — confirm expected content is ON SCREEN
 4. **COMMIT:** Save to docs/qa/<feature>/helix_ota---<feature>.mp4
-

H-08 [AGENT] — Independent Code Review → Zero-Finding GO

Effort: M (~2h) **Source:** Constitution §11.4.165, §11.4.142

Process:

1. **Independent verifier agent** reviews ALL new/changed code from Stages B, C, D, E, F, G.
2. Verifier iterates: finding → fix → re-verify → repeat until zero findings.
3. Each round: the verifier is structurally SEPARATE from the generator (different agent session).
4. Final state: zero-finding GO — no bugs, no style violations, no security issues, no test gaps.

Review scope:

- Every file changed in feature/multi-tenant-accounts (Accounts)
 - Every file changed in server hardening (Stage B)
 - Every file changed in submodules/website (Website)
 - Every file changed in device-side (Stage E)
 - Every deploy script, compose file, nginx config
 - Every migration SQL file
-

H-09 [OPERATOR] — §11.4.185 Manual QA Team Final Confirmation

Effort: L (operator + QA team time) **Source:** Constitution §11.4.185 — MANDATORY FINAL GATE

What the QA team must manually confirm:

On the production server: 1. Login as admin, create a new account 2. Create a project, add members with different roles 3. Upload a signed OTA artifact 4. Create a release, deployment, and staged rollout 5. Verify device registration works 6. Verify audit logs capture all actions 7. Verify webhook dispatches work 8. Verify rate limiting works under load

On the physical RK3588 device: 1. Flash the system image with OTA agent pre-installed 2. Device boots, OTA agent registers with server 3. Device discovers an available update 4. Device downloads, verifies, and applies the update 5. Device reboots into the new slot successfully 6. Device reports telemetry back to server 7. Deploy a deliberately faulty OTA — device auto-rollbacks

On the dashboard: 1. Login, navigate all pages 2. Verify device list shows the registered devices 3. Verify telemetry charts show real data 4. Verify deployment progress updates in real-time

On the website: 1. Visit <https://hxota.com> 2. Verify all pages load, content is correct 3. Verify responsive on mobile/tablet/desktop 4. Verify dark/light theme toggle works

Acceptance: QA team signs off. THIS IS THE FINAL GATE.

H-10 [AGENT] — Tag Release

Effort: S (~15 min) **Source:** §11.4.151 (release tagging)

What to do (ONLY after H-09 PASS):

1. **Determine version:** If Accounts (Stage C) was completed, this is 2.0.0. If only hardening was done, 1.1.0.

2. **Tag main repo:**

```
git tag -a helix_ota-2.0.0 -m "Helix OTA 2.0.0 – Multi-tenant
Accounts + production deployment"
git push github helix_ota-2.0.0
git push gitlab helix_ota-2.0.0
git push gitflic helix_ota-2.0.0
git push gitverse helix_ota-2.0.0
```

3. **Tag each touched submodule** with the SAME prefix:

```
for sub
    in submodules/ota-protocol submodules/ota-android-
agent submodules/website; do
    cd $sub
    git tag -a helix_ota-2.0.0 -m "Helix OTA 2.0.0"
    git push origin helix_ota-2.0.0
    cd ../../..
done
```

4. **Update RESUMPTION.md, Fixed.md, Issues.md:**

- Move any closed items to Fixed.md
- Update RESUMPTION.md to reflect 2.0.0 state

- Update AGENTS.md/GEMINI.md/CLAUDE.md/QWEN.md carrier metadata

5. Push all tracking doc updates:

```
bash scripts/commit_all.sh "release: helix_ota-2.0.0"
```

Final Acceptance Checklist

Step	Action	Result
H-01	Server full test suite	All GREEN, 0 races
H-02	OTA bricks test suite	All GREEN, 0 crashes
H-03	E2E pipeline against production	All PASS
H-04	Android agent real-device	All GREEN
H-05	DDoS resilience	Server survives, rate limiter works
H-06	Web-surface tests	All GREEN, a11y clean
H-07	Video recordings	All features recorded + verified
H-08	Independent code review	Zero-finding GO
H-09	Manual QA confirmation	QA TEAM SIGN-OFF
H-10	Release tagged	Tag pushed to all 4 upstreams

Honest Boundary (§11.4.6)

- H-09 (§11.4.185) is the SINGLE non-negotiable gate. An agent CANNOT self-certify it.
- The manual QA team must perform their confirmation on the REAL production deployment with REAL hardware.
- Until H-09 passes, the system is “development-complete” but NOT “released.”
- H-10 is the mechanical tagging step — it takes 15 minutes but CANNOT run before H-09.